



**NANYANG
TECHNOLOGICAL
UNIVERSITY**
SINGAPORE

Semester Group Project

by

Name	Matric No.	Work %
Timothy Chang	<i>U2220136J</i>	50.0%
Neo Zhi Xuan	<i>U2222293E</i>	50.0%

Matric (query source)	A5656567B	Target year	2016
Start month	6 (June)	Threshold	≤ 4725
Town list	JURONG WEST, CLEMENTI, HOUGANG, PASIR RIS, TAMPINES, YISHUN, BEDOK		
Ranges	$x \in [1, 8], y \in [80, 150] \Rightarrow 568 (x, y)$ query pairs		

SC4023 Big Data Management

NANYANG TECHNOLOGICAL UNIVERSITY

April 2026

Abstract. We present our highly performant columnar store query engine. It implements **14 distinct optimisations**, spanning five architectural layers. They include encoding (Dictionary, Precomputed aggregates), Physical layout (Pre-sorted clustering, Per-column binary files, Town partitioning), Indexing (Zone maps, Bitmap indices, Binary search), Scan-path rewriting (Result reuse, Late materialisation, Predicate reordering, Integer gating), and I/O management (Chunked streaming, Memory-mapped I/O). All optimisations are toggleable by boolean flags. This enables us to build a combinatorial evaluation suite testing across 40 permutations of optimisations, each with correctness verification. Physical town partitioning together with Result reuse achieves a **2,013× query speedup** at only 660 KB memory. Memory-mapped I/O with Result Reuse has the fastest end to end time of **84.8 ms**, 58.5× faster than baseline. Negative results are instructive as well. Bitmap indexing (0.08×) and Predicate reordering (0.03×) show that remove cheap predicates via an expensive process greatly affects performance, especially when data is already well filtered upstream.

1 Data Storage

1.1 System Architecture

Figure 1: Pipeline: CSV → parser → per-column binary files → engine → output CSV.

Our parser detects column positions from the CSV header and streams rows into parallel `std::vector` columns. YYYY-MM date strings are split into Year (`uint16`) and Month (`uint8`). Invalid rows (malformed dates, empty fields, etc) are skipped.

1.2 Baseline Column Store

The `ColumnStore` struct holds one `std::vector` per column. Numeric columns use fixed-width types as seen in Table 1. All columns are populated in one pass through the CSV. For the baseline, loads of all 10 columns into memory produces a total footprint of 71.4 MB.

1.3 Per-Column Physical Layout

Column	Type	Encoding	B/row	Derived?	Rationale
Year	<code>uint16</code>	raw	2	from YYYY-MM	range predicate
Month	<code>uint8</code>	raw	1	from YYYY-MM	range predicate
Town	<code>uint8</code>	dict (A1)	1	no	26 towns ≤ 5 bits
Floor_Area	<code>uint16</code>	raw	2	no	≥ y predicate
Resale_Price	<code>uint32</code>	raw	4	no	aggregate target
PricePerSqm (A4)	<code>float32</code>	raw	4	yes	removes per-row division
Block, Flat_Model, Lease_Commence	...	late-materialised (§2.5)			output only

Table 1: Physical layout per column.

1.4 Encoding and Layout Decisions

Dictionary encoding (A1). A `DictionaryEncoder` (bidirectional `unordered_map/vector` pair) maps each of 26 unique town strings to a `uint16_t` ID. At query time, the engine pre-resolves the target town list to IDs once outside the scan loop, then uses integer equality in the inner loop. Measured: 34% query speedup (2,405→1,585 ms) at the cost of +2.9% memory and 15.5% slower load time for dictionary construction—a one-time penalty amortised across 568 queries.

Column splitting of Month. Splitting the date string into two integer columns at parse time eliminates repeated string parsing and enables direct `uint16` range predicates. The Year predicate alone eliminates ~90% of rows with a single integer comparison.

Precomputed PricePerSqm (A4). Storing price / area as `float32` eliminates a per-row floating-point division for filter survivors. However, only ~0.2% of row iterations reach this division (~543 survivors per query), so the measured speedup is <0.1%. The column is retained because the query’s aggregate target *is* price-per-sqm, but the benefit is workload-dependent.

Pre-sorted clustering (A2). A `stable_sort` by composite key (Town, Year, Month) physically reorders all columns once at ingest using a shared permutation vector. A linear post-sort pass records each town partition’s [begin, end) boundaries for $O(1)$ lookup at query time. This is the foundation for binary search (§2.4) and temporally homogeneous zone-map chunks (§1.5).

Separate binary column files (A9). Each column is written to its own binary file during CSV-to-columnar conversion. At query time only the five filter columns are loaded; output columns are lazy-loaded only for the

single surviving row per (x, y) pair. Memory drops from 71.4 MB to 2.7 MB (96.2% reduction) and load time from 2,553 ms to 114 ms. A9 alone does *not* improve query time — it is 2.1% *slower* because the binary I/O path still performs a full scan. A9’s value is in memory and load-time reduction when composed with scan-elimination techniques.

1.5 Zone Maps (B1)

Each numeric filter column is divided into fixed-size chunks of 1,024 rows, and a per-chunk (min, max) entry is stored at ingest—254 chunks totalling <8 KB. With sorted data (A2), temporal locality maximises chunk homogeneity, achieving a 96.1% skip rate across all 568 queries (138.6K of 144.3K chunks skipped).

1.6 Bitmap Index on Town (B2)

One `std::vector<bool>` is built per distinct town at load time (26 bit-vectors $\times$ 259K bits $\approx$ 842 KB). Before the query loop, the bit-vectors for the target towns are OR-combined into a single `uint8_t`-per-row mask ($\sim$ 259 KB), replacing the per-row town predicate with a single array lookup: `if (!mask[i]) continue`. The mask uses one byte per row instead of packed bits for cache-line-friendly sequential access in the scan loop. Benchmark results (§3.5) show this is a *negative* optimisation: eliminating all 12.55M town comparisons yielded 0.08 $\times$ because the per-row byte-load-and-branch adds overhead that exceeds the original integer comparison cost—the cheap Year check already prunes $\sim$ 90% of rows before Town is reached.

1.7 I/O Layer: Chunked (D1) and Memory-Mapped (D2)

Chunked I/O (D1). To address the “larger than memory” requirement (clarification Q4), the system streams filter columns from A9 binary files in bounded-size chunks under a configurable memory budget. The outer loop iterates over I/O chunks while the inner loop runs all 568 queries per chunk (loop inversion), exploiting the associativity of min to merge per-chunk winners.

Memory-mapped I/O (D2). D2 replaces `ifstream/fread` with POSIX `mmap(2)`: each `.col` file is mapped into virtual address space with a single syscall, then `memcpy`’d into the existing column vectors. This is a *memcpy-from-mapped-region* approach—not true zero-copy—chosen for full compatibility with downstream optimisations. Load time drops from 114 ms (A9 `ifstream`) to 63 ms (45% reduction). For lazy materialisation, `loadStringAtMmap` replaces `std::ifstream` construction with raw POSIX calls, avoiding C++ `iostream` overhead on the per-winning-row path. D2 conflicts with D1 (opposite I/O philosophies: demand-paged vs. explicitly bounded) and is disabled if both are set. Composed with reuse, D2+C1C2 achieves the fastest total time of **84.8 ms** (58.5 $\times$ end-to-end).

1.8 Physical Partitioning by Town (E1)

E1 splits columnar binary files into one subdirectory per town (Hive-style partition pruning). At query time, `loadColumnFilesPartitioned` opens only the subdirectories for the target towns and concatenates their columns into the in-memory `ColumnStore`:

Listing 1: E1 partition loading (simplified).

```

1 void loadColumnFilesPartitioned(const string& base_dir,
2   const vector<string>& target_towns, ColumnStore& db) {
3   for (const auto& town : target_towns) {
4     string dir = base_dir + "/" + town;
5     // append filter + materialisation columns from dir
6     appendNumericCol(db.col_month_year, dir+"/month_year.col");
7     appendNumericCol(db.col_floor_area, dir+"/floor_area.col");
8     // ... remaining columns ...
9   }
10  // rebuild zone maps / bitmap indices over concatenated data
11 }

```

For 3 target towns, only 42,935 of 259,237 rows are loaded—an 83.4% reduction. The Town predicate becomes implicit: if a row is in memory, it belongs to a target town; zero town comparisons occur at scan time. Materialisation columns are loaded eagerly (not lazy), trading higher memory for eliminating per-query file I/O. Memory footprint drops to **660 KB** (99.1% less than baseline) because only target-town rows are resident.

Planned extension: Run-length encoding (RLE) on the sorted Town column within each partition, exploiting contiguous runs created by A2 to compress `town_encoded.col` to a single (value, count) pair per partition.

1.9 Exception Handling

Empty result sets produce a “No result” row in the output CSV. Ties in minimum price-per-sqm are resolved by first-encountered order (clarification Q5: no penalty).

1.10 Space–Time Trade-off Summary

Decision	$\Delta\text{Mem} / \Delta\text{Load}$	ΔQuery	Verdict
Dict-encode Town (A1)	+2.9% mem, −15.5% load	−34% query	keep
Precompute PricePerSqm (A4)	+2.8% mem	−0.1% query	keep (negligible)
Pre-sort (Town,Y,M) (A2)	+151.8% load	enables B3	keep (foundation)
Columnar files (A9)	−96.2% mem, −95.5% load	+2.1% query	keep (I/O win)
Zone maps (B1)	+ ϵ mem	−21% query	keep
Bitmap index Town (B2)	+1.1% mem	−1222% query	reject
mmap I/O (D2)	~ mem	−45% load	keep (I/O)
Town partitioning (E1)	−99.1% mem	−85.5% query	keep
RLE Town column (planned)	− ϵ disk	TBD	placeholder

Table 2: Storage decisions as explicit trade-offs. All deltas vs. baseline.

2 Data Processing

2.1 Baseline Scan

The baseline scans all 259,237 rows for each of 568 (x, y) pairs. Predicates are evaluated in the order Year $\rightarrow$ Month $\rightarrow$ Town $\rightarrow$ Area; Year is checked first because it eliminates $\sim 90\%$ of rows with a single `uint16` equality before the more expensive Town iteration runs.

Listing 2: Baseline scan loop in `runQuery` (simplified).

```

1 for (size_t i = 0; i < N; ++i) {
2     if (db.col_month_year[i] != target_year) continue; // ~90% eliminated
3     if (db.col_month_month[i] < start_month ||
4         db.col_month_month[i] > end_month) continue;
5     // Town: linear scan over 7-element target list
6     bool match = false;
7     for (const auto& t : towns)
8         if (db.col_town[i] == t) { match = true; break; }
9     if (!match) continue;
10    if (db.col_floor_area[i] < y) continue;
11    double ppsm = (double)db.col_resale_price[i] / db.col_floor_area[i];
12    if (ppsm < min_ppsm) { min_ppsm = ppsm; best_i = i; }
13 }
```

Across the full grid this produces 147.25M row iterations and 12.55M town comparisons, averaging 4,957.7 ms total (2,552.9 ms load + 2,404.8 ms query).

2.2 Composable Scan Architecture

All 14 optimisations are controlled by independent boolean flags in the `ColumnStore` struct (toggled via `OptConfig` in the evaluation suite). The query engine has three paths: (1) a *reuse path* (C1+C2) that short-circuits via the cumulative table (§2.3), (2) a *unified scan loop* where each flag controls one orthogonal decision—A1 selects int vs. string comparison, B1 wraps the loop in chunk-level pruning, B2 replaces the town predicate with a bitmap mask, C4 reorders predicates, C6 gates float computation, A4 selects the PPSM source, and C3 defers the price column to a Phase 2 survivors pass, and (3) a *partition-pruned path* (E1) that loads only target-town data before entering either path (1) or (2). D2 (mmap) is an I/O-layer substitution under A9, transparent to the scan logic. This composability allows the evaluation suite to test ~ 40 configurations and isolate each contribution.

2.3 Result Reuse (C1+C2)

The 568 queries overlap: for fixed y , the month window for $x=k$ is a superset of $x=k-1$; for fixed x , every record with $\text{area} \geq y$ also satisfies $\text{area} \geq y-1$.

Three-step cumulative table build. Step 1 (single $O(N)$ scan): For each row passing year and town filters, compute $\text{offset} = \text{month} - \text{start} + 1$ (range 1–8) and $\text{bucket} = \min(\text{area}, 150)$. Records with $\text{area} > 150$ are *clamped* to bucket 150, not dropped—this is correct because every valid y threshold is ≤ 150 , so a record with area 160 satisfies every y and must contribute to all buckets it qualifies for; the clamped bucket propagates into all lower buckets during the Y-sweep.

Listing 3: `buildCumulativeTable` — Step 1 scan (core loop).

```

1 vector<vector<MinEntry>> per_x(9, vector<MinEntry>(151));
2 for (size_t i = 0; i < N; ++i) {
3     if (db.col_month_year[i] != target_year) continue;
4     uint8_t month = db.col_month_month[i];
5     if (month < start_month) continue;
6     int offset = (int)month - (int)start_month + 1;
```

```

7  if (offset < 1 || offset > 8) continue;
8  // town filter (A1-aware: int vs string)
9  if (db.use_dict_encoding) {
10     bool match = false;
11     for (auto& tid : town_ids)
12         if (db.col_town_encoded[i] == tid) { match = true; break; }
13     if (!match) continue;
14 } else { if (town_set.find(db.col_town[i]) == town_set.end()) continue; }
15 unsigned area = db.col_floor_area[i];
16 if (area < 80) continue;
17 unsigned bucket = (area > 150) ? 150u : area; // clamp, NOT drop
18 double ppsm = (double)db.col_resale_price[i] / db.col_floor_area[i];
19 if (!per_x[offset][bucket].has || ppsm < per_x[offset][bucket].ppsm)
20     per_x[offset][bucket] = {true, ppsm, i};
21 }

```

Step 2 (X-sweep): For each area bucket, sweep offsets $1 \rightarrow 8$, propagating a running minimum. **Step 3 (Y-sweep):** For each offset, sweep area $149 \rightarrow 80$, propagating the minimum downward. After Step 3, `cum_table[x][y]` holds the answer for each (x, y) pair, and the query phase becomes a single array lookup.

Results. C1+C2 alone reduces query time from 2,404.8 ms to 4.6 ms ($528\times$ query speedup), with rows scanned dropping from 147.25M to 259.2K (the single build scan). When composed with dictionary encoding (A1+C1+C2), query time drops to 3.3 ms ($721\times$).

2.4 Pre-sort + Binary Search (A2+B3)

With data pre-sorted by (Town, Year, Month), the query engine runs custom `lowerBound/upperBound` searches within each town partition using a linearised month key ($\text{year} \times 12 + \text{month} - 1$):

Listing 4: Binary search on linearised month key.

```

1  inline uint32_t monthKey(uint16_t year, uint8_t month) {
2      return (uint32_t)year * 12u + (uint32_t)(month - 1u);
3  }
4  size_t lowerBoundMonthKey(const ColumnStore& db, size_t l, size_t r,
5                          uint32_t key) {
6      while (l < r) {
7          size_t mid = l + (r - l) / 2;
8          if (monthKeyAt(db, mid) < key) l = mid + 1; else r = mid;
9      }
10     return l;
11 }

```

Rows scanned drop from 147.25M to 751.7K (99.5% reduction), town comparisons fall to zero, and query time drops to 16.3 ms ($148\times$ query speedup). This is an independent scan-elimination path from C1+C2.

2.5 Late Materialisation (C3)

Phase 1 applies only the four predicate columns, appending survivor indices to a vector. Phase 2 iterates survivors to fetch `Resale_Price`, compute `PricePerSqm`, and track the minimum. Measured result: 103% *slowdown* (4,888 ms vs. 2,405 ms), because the overhead of maintaining the survivor vector exceeds cache savings from deferring a single 4-byte column.

2.6 Other Scan Optimisations

Zone-map chunk skipping (B1). Before scanning a 1,024-row chunk, the engine tests its min/max against Year, Month, and Floor_Area predicates; failing chunks are skipped entirely (96.1% skip rate, 21% standalone query speedup; combined with A1: 20.8%).

Bitmap-driven row filtering (B2). A precomputed `uint8_t` mask replaces the per-row town loop with a single byte lookup. Town comparisons drop to zero, but query time regressed to $0.08\times$ (31,782 ms vs. 2,405 ms baseline): the byte-load-plus-branch on every row is more expensive than the original integer comparison that runs only on the $\sim 10\%$ of rows surviving the Year filter. This confirms that the baseline Year $\rightarrow$ Month $\rightarrow$ Town order is already near-optimal.

Predicate reordering (C4). Reordering to Town $\rightarrow$ Year $\rightarrow$ Month $\rightarrow$ Area caused a $0.03\times$ regression: town comparisons surged from 12.55M to 417.51M because every row now pays the expensive Town list iteration before the cheap Year check can eliminate it. The lesson: optimal order must minimise *cost* $\times$ *pass_rate*, not maximise selectivity alone.

Integer early-exit gate (C6). Tests $\text{price} \times \text{cur_min_area} \geq \text{cur_min_price} \times \text{area}$ in uint64 before computing float PricePerSqm. Measured: within noise—too few survivors ($\sim 543/\text{query}$) for the eliminated float work to register.

2.7 Correctness Argument

Each optimisation preserves output equivalence: **A1** dictionary encoding is a bijection. **A2** sorting is a permutation. **A4** precomputed PPSM is identical to runtime division. **A9** binary files reproduce identical column vectors. **B1** zone maps skip a chunk only when its (min, max) proves no row can match. **B2** bitmap mask is the OR of exact per-town bit-vectors; the mask lookup is equivalent to the membership test it replaces. **B3** binary search locates exact boundaries in sorted data. **C1+C2** result reuse is exact because min is associative over disjoint month ranges and monotone over area thresholds; clamping $\text{area} > 150$ to bucket 150 is correct because all valid $y \leq 150$. **C3** defers column access without altering the predicate set. **C4** changes evaluation order, not the predicates. **C6** is a sufficient condition for “cannot beat current minimum.” **D1** processes the same rows in bounded batches; the cumulative table is merged across chunks via the associativity of min. **D2** memcpy from mapped region produces byte-identical vectors as fread. **E1** partition loading concatenates disjoint town subsets; the union equals the set of rows matching the town predicate. All ~ 40 configurations produced byte-identical output, verified by the evaluation suite’s per-result hash comparison (§3.4).

3 Experiment Result

3.1 Compulsory Artefacts

(a) End-to-end build + run.

(b) ScanResult_A5656567B.csv.

Figure 2: Execution screenshots (spec-mandated).

3.2 Excel Cross-Validation

Engine outputs were validated against Excel MINIFS/FILTER for representative (x, y) pairs. Table 3 confirms byte-exact agreement across different selectivities; additional pairs are in the Appendix.

x	y	Engine PPSM	Excel PPSM	Match
1	80	[value]	[value]	✓
4	100	[value]	[value]	✓
7	130	[value]	[value]	✓
8	150	[value]	[value]	✓

Table 3: Excel cross-validation for selected (x, y) pairs.

3.3 Benchmark Table

Each configuration was benchmarked; memory is estimated from column vector capacities plus dictionary/zone-map overhead. Table 4 shows a cumulative build-up revealing three independent optimisation paths: result reuse (C1+C2), pre-sort + binary search (A2+B3), and physical town partitioning (E1).

Configuration	Load (ms)	Query (ms)	Total (ms)	Q.Speedup	Rows Scan.	Memory
Baseline	2,552.9	2,404.8	4,957.7	1.0×	147.25M	71.4 MB
+ A1 Dict Encoding	2,948.1	1,585.2	4,533.3	1.5×	147.25M	73.4 MB
+ C1+C2 Reuse (alone)	2,510.1	4.6	2,514.7	528×	259.2K	71.4 MB
+ A1+C1+C2 Dict+Reuse	2,950.9	3.3	2,954.2	721×	259.2K	73.4 MB
+ A9+C1C2 Columnar+Reuse	82.6	57.6	140.2	41.7×	259.2K	2.7 MB
+ A9+All Everything	171.0	122.2	293.1	19.7×	259.2K	5.3 MB
<i>Memory-mapped I/O path (D2):</i>						
D2+C1C2 mmap+Reuse	38.1	46.7	84.8	51.5×	259.2K	5.3 MB
D2+All mmap+Everything	72.6	65.4	138.0	36.8×	259.2K	6.1 MB
<i>Alternative scan-elimination path (A2+B3):</i>						
A2+B3 Pre-sort+BSearch	3,932.4	16.3	3,948.6	148×	751.7K	71.0 MB
A9+A2+B3 Col+Sort+BSearch	175.6	117.8	293.4	20.4×	751.7K	2.7 MB
<i>Partition-pruned path (E1):</i>						
E1: Town Partition	122.8	348.8	471.7	6.9×	24.4M	660 KB
E1+B1 Partition+ZoneMaps	168.0	297.2	465.3	8.1×	24.4M	1.2 MB
E1+C1C2 Partition+Reuse	149.6	1.2	150.8	2,013×	42.9K	660 KB
E1+All Partition+Everything	151.8	1.1	153.0	2,127×	42.9K	1.2 MB

Table 4: Benchmark across key configurations. Q.Speedup = baseline query time / config query time.

Figure 3: Speedup vs. baseline (= 1.0×) across key configurations.

3.4 Correctness Invariant

The evaluation suite (`eval_suite.cpp`) compares each configuration’s 568-entry output vector against the baseline element-by-element, checking that the (x, y) pair, all output fields, and the computed PricePerSqm are identical. All ~40 configurations report PASS.

3.5 Negative and Neutral Results

The professor explicitly requested trade-off justification (clarification Q1). Table 5 isolates five optimisations that underdelivered. C4 and B2 are the most instructive: both are textbook-recommended techniques that regressed dramatically because they violated `cost×pass_rate` ordering.

Configuration	Query (ms)	Δ vs Baseline	Verdict
C4: Predicate Reorder	72,963.5	0.03×	Rejected
B2: Bitmap Town Index	31,782.3	0.08×	Rejected
C3: Late Materialise	4,887.7	0.49×	Kept for architecture
A4: Precompute PPSM	4,675.6	0.51×	Within noise
C6: Int Multiply Gate	3,692.7	0.65×	Within noise

Table 5: Negative and neutral results (individual isolation).

B2 is instructive alongside C4: both attempt to eliminate town comparisons, yet B2 *regressed* because the per-row byte-load adds branch overhead on all 147M row-visits, while A2+B3 avoids visiting non-target rows entirely. C3 underperformed because deferring a 4-byte column on a narrow schema yields less cache savings than the overhead of maintaining a survivor index vector.

3.6 Sensitivity Analysis: D1 Memory Budget

Table 6 varies the D1 memory budget across a 50× range. All D1 configurations use A9 columnar files with A1, B1, and result reuse built into the batch runner.

Budget	I/O Chunks	Query (ms)	Total (ms)	Q.Speedup
50 MB	1	719.5	778.4	3.34×
10 MB	1	678.2	730.3	3.55×
5 MB	2	685.3	739.5	3.51×
2 MB	3	724.3	767.7	3.32×
1 MB	6	777.1	820.7	3.09×

Table 6: D1 chunked I/O sensitivity. Total bytes read = 4.7 MB at all budgets.

Performance is stable from 50 MB down to 2 MB, degrading 8% at the 1 MB extreme where 6 I/O chunks are required. Byte-identical correctness is maintained at every budget.

3.7 Environment

All benchmarks were run on an Apple M-series MacBook Air, compiled with `clang++ -O2 -std=c++17`. The dataset contains 259,237 rows across 10 columns; all 568 (x, y) query pairs were evaluated per configuration.

4 Conclusion

The largest query-time win was result reuse (C1+C2): a single $O(N)$ build pass replaced $568 \times O(N)$ scans with $O(1)$ lookups (528× query speedup; 721× with A1). Physical town partitioning (E1) reduced the build-pass input from 259.2K to 42.9K rows, yielding 2,013× query speedup and 660 KB memory—the most memory-efficient configuration. The fastest end-to-end configuration was D2+C1C2 (mmap + reuse) at 84.8 ms total (58.5×), where mmap’s 45% load-time reduction composed with reuse’s scan elimination. The most instructive failures were C4 predicate reordering (0.03×) and B2 bitmap indexing (0.08×): C4 demonstrated that optimal predicate order must minimise `cost × pass_rate`, while B2 showed that eliminating a cheap predicate via a more expensive mechanism can regress performance when upstream predicates already prune effectively. Across all 14 optimisations and ~40 tested configurations, every result was byte-identical to the baseline—correctness was never sacrificed for speed. If revisiting this project, we would explore RLE compression on sorted partition columns and a cost-based predicate optimiser that dynamically selects evaluation order.

Appendix: Figures (≤ 2 pages)

Figure 4: Full pipeline: CSV → parser → per-column binary files → query engine → output CSV. Derived columns (Year, Month, PricePerSqm) shaded; lazy-loaded output columns (Block, Flat_Model, Street_Name) dashed.

Figure 5: Extended cross-validation for additional (x, y) pairs. All engine outputs match Excel MINIFS results.

Figure 6: Speedup vs. baseline across all configurations. Three independent scan-elimination paths visible: C1+C2 result reuse (721 $\times$), A2+B3 pre-sort+binary search (148 $\times$), and E1+C1C2 partition+reuse (2,013 $\times$).

Figure 7: Pseudocode for `buildCumulativeTable`: Step 1 single $O(N)$ scan, Step 2 X-sweep across month offsets, Step 3 Y-sweep from higher to lower area thresholds.